

Leakage-Safe Pregame Pitcher Strikeout Projection from Baseball Savant

A quant ML engineering study of rate modeling, exposure projection, and governed decisioning

Cameron Kaplinger
Independent Researcher

Technical manuscript · Updated Aug 2026

Code repository: <https://github.com/Cbkaplinger/MLB-Props>

Acknowledgments. Baseball Savant / Statcast pitch-level data provided the empirical foundation for this work.

Abstract

This paper presents a leakage-safe **pregame** machine learning pipeline for starting-pitcher strikeout projection from Baseball Savant (Statcast) pitch-level data. The core target is game-level strikeout rate $k_rate = K / PA$ for starters who face at least nine batters, using only pregame features. A Polars-first three-level pipeline builds game aggregates, lagged rolling form, and a model-ready training frame under strict chronological validation.

The active production lane uses a **two-model LightGBM blend** across frozen feature sets (`production_sparse72_monotone`, `production_final58_consensus`) with weights `0.60 / 0.40`, plus a Ridge projected-TBF companion [2]. Count projections follow $E[K] = k_rate_hat \times TBF_hat$, then convert to line probabilities via binomial/Poisson on projected exposure only [3, 4]. Governance is run through deduped replay + transfer calibration, where the active deployment profile reports ROI `0.4363`, PnL `+24.17u` (`1u = 50 USD`), Sharpe `0.4352`, Sortino `0.4277`, and positive market-skill deltas (`+0.2069` Brier, `+0.1551` LogLoss) on artifact-backed evaluation.

Trial-adjusted significance testing reports a Deflated Sharpe Ratio (DSR) of `0.0349`, indicating that the current 26-bet audited sample is still underpowered for a strong post-selection edge claim.

The main contribution is an end-to-end quant workflow that links leakage-safe modeling, chronological evaluation, and governed decision operations in a reproducible system.

1. Introduction

Strikeout props are a natural target for pregame modeling: the outcome is well-defined, Statcast supplies rich pitch- and PA-level detail, and the quantity of interest separates into a **rate** component and an **exposure** component. Many published baseball analytics workflows emphasize descriptive leaderboards or postgame attribution. Betting-oriented systems often blur the pregame information set. This work treats the problem as **supervised prediction under a strict pregame constraint**.

The modeling claim is simple and compositional. A leakage-safe estimate of strikeout rate, multiplied by a leakage-safe projection of batters faced, yields expected strikeouts and line probabilities without ever using same-game outcomes as inputs:

$$k_rate \times TBF \rightarrow E[K] \rightarrow P(K \geq L)$$

Goal. Estimate a starter's strikeout rate before first pitch, project how many batters that starter will face, and convert the pair into expected strikeouts and $P(K \geq L)$ for common prop lines L .

Estimand. Research metrics use the $PA \geq 9$ cohort defined in Section 3.3.

Figure 1. Leakage-safe architecture. Raw Statcast pitches are aggregated into game records (Level 1), lagged rolling form (Level 2), and a model-ready training frame (Level 3). A LightGBM strikeout-rate model and a Ridge projected-TBF model combine in a count layer that yields expected strikeouts and line probabilities from projected exposure only.

1.1 Related work

Sabermetric rate-based pitching models. Fielding Independent Pitching (FIP) and related estimators such as xFIP summarize pitcher skill from strikeouts, walks, hit batsmen, and home runs (or home-run rates normalized by fly-ball environment), reducing dependence on balls in play and defensive context [7, 8]. Those metrics are primarily descriptive or talent-estimation tools at the season or large-sample level. The present work is complementary: it retains FIP/xFIP-style components as *candidate features*, but the prediction target is game-level `k_rate` under an explicit pregame information constraint, not a restatement of FIP as the forecast.

Season-level baseball projection systems. Systems such as Marcel [9], PECOTA [10], Steamer, and ZiPS forecast season (or rest-of-season) player rates from weighted recent performance, regression to the mean, aging, and—depending on the system—comparable-player paths. They are useful conceptual baselines for talent estimation, while this manuscript focuses on production pregame decision governance under chronological constraints.

Chronological evaluation and leakage control. When targets are ordered in time, randomly reshuffled cross-validation overstates accuracy by allowing future information into training folds [11]. Forecasting practice therefore prefers expanding or rolling windows and features that are known at the forecast origin. This paper treats those constraints as hard engineering rules (shifted rolling windows, prior-season park factors, date-disjoint partitions) and verifies them with tests and audits rather than as an after-the-fact caveat.

Count models for rate $\times$ exposure. Once a mean rate and an exposure (here, projected batters faced) are specified, Poisson or binomial probabilities are standard for count outcomes [3, 4]. Line probabilities use those trials on *projected* exposure only. A beta-binomial dispersion check collapses to the binomial limit under the frozen mean, consistent with a well-specified mean model absorbing extra-binomial variance [4].

2. Contributions

1. **Leakage-safe feature architecture.** Same-game outcomes never enter predictors; rolling statistics are shifted; park factors use prior seasons only; chronological splits never divide a calendar date across partitions.
2. **Frozen multi-set rate modeling.** The rate lane is now governed across three frozen feature sets (`sparse72`, `sparse72_monotone`, `final58`) with explicit champion/challenger workflow and artifact lineage.
3. **Dual-layer projection stack.** A weighted LightGBM ensemble for `k_rate_hat`, Ridge for projected TBF, and a count layer on projected exposure produce expected strikeouts and line probabilities.
4. **Quant governance integration.** Open-universe skill ranking, deduped manual replay, isotonic transfer calibration, and board-to-ledger parity checks are wired into the daily production decision path.
5. **Reproducible operations.** Policy profiles, calibration pointers, and model lineage are versioned so retraining, promotion, and daily execution are auditable.

3. Data and pipeline

Because the stack multiplies rate by exposure, both quantities must be built from the same leakage-safe information set. The pipeline below is the shared foundation for that claim (Figure 1).

3.1 Source

Pitch-level regular-season Statcast via Baseball Savant (local parquet cache, seasons 2015–2026 retained for coverage; **model fitting uses 2023–2024 rows**). Season 2022 supplies prior-only park and league context for 2023 boundaries and does not enter training rows. Postseason files are retained but not used in the strikeout stack documented here.

3.2 Three levels

Table 1. Three-level pipeline outputs.

Level	Role	Primary outputs
1 · Games	Pitch → starter/batter game aggregates	pitcher_games, batter_games, pitch_type_games, park_factors
2 · Rolling	Leakage-safe lagged form + context	pitcher_rolling, batter_rolling
3 · Training	Join lineup + park into model frame	pitcher_training, batter_training

Level 1 is the audit surface: denominators, events, and identities are defined once. Level 2 applies rolling and season-to-date windows with an explicit lag so the game being predicted never contributes to its own features. Level 3 assembles opponent-lineup aggregates and prior-season park factors.

Implementation is Polars-first, with automated tests for feature safety, pipeline stages, and the trainer/splitter. Repository paths and artifact names are collected in Appendix A.

3.3 Population filter

Default research rows require $PA \geq 9$. This is a **postgame** cohort definition for a **pregame** model: it does not leak feature values, but it conditions every reported metric. Population audits show excluded share $\approx 3.5\%$ (2023–2024). Cutoffs 8–10 change exclusion by about half a percentage point; nine remains the frozen policy.

4. Leakage methodology

Leakage control is not a preamble to the rate $\times$ exposure claim—it is what makes the claim scientifically meaningful. If same-game outcomes contaminate features, both the rate model and the TBF model become postgame reconstructions rather than pregame forecasts.

The following rules are treated as hard constraints:

- Same-game K, PA, Outs, and `k_rate` are labels / evaluation fields only.
- Rolling and season-to-date player statistics are shifted by one game or start.
- Season-to-date windows reset at season boundaries.
- Park factors for season Y use only seasons before Y.
- Opponent lineup aggregates use each batter’s **pregame** form; historical membership is the first nine distinct batters by first PA.
- Train / validation / test splits are chronological; a calendar date lies in exactly one partition.
- Unexpected numeric columns are rejected unless they match approved pregame naming rules.

Verification combines notebook spot checks (first start of season, season boundary resets, manual rolling recomputation) with an automated test suite. Process bugs (for example relocated-park blending; Section 9) were logged with before/after evidence in the research log.

Evaluation scope. Development metrics use 2023–2024 chronological partitions and nested folds. Any 2025 reporting is treated as non-selection context rather than a pristine post-freeze holdout.

5. Feature design

With the information set fixed, feature design is treated as a quant selection problem: keep only pregame signals that survive chronological evaluation and operational governance checks.

5.1 Active feature sets

The rate lane is maintained through three frozen sets:

- `production_sparse72` (compact baseline),

- `production_sparse72_monotone` (same sparse spine with monotone constraints),
- `production_final58_consensus` (consensus-pruned compact set).

These sets are evaluated both as individual models and as ensemble members.

5.2 Monotone-constraint implementation scope

Monotone constraints are implemented in the LightGBM production lane because that path is hardened in the current training/evaluation stack [1]. XGBoost also supports monotonic constraints, and Aug 2026 parity runs include explicit constrained-vs-unconstrained XGBoost comparisons.

5.3 Model selection criteria

Production promotion decisions are driven by:

1. feature-level pruning outcomes on frozen sparse sets,
2. rolling-window sensitivity checks,
3. out-of-sample market-skill governance lanes (open and deduped manual),
4. deployment robustness (calibration transfer and parity checks).

Family ablation serves as a challenger screen, while production promotion is determined in the governance lane.

5.4 Empirical-Bayes style shrinkage in features

The production feature pipeline uses empirical-Bayes style shrinkage selectively to stabilize low-sample pregame rates:

- `src/Python/batter_rolling.py`: batter rolling K% shrinkage (`k_rate_std_shrunk`) toward batter prior + league prior with pseudo-PA strength.
- `src/Python/pitcher_rolling.py`: prior-season shrunk pitcher K/PA (`add_prior_season_shrunk_k`) and low-sample pitch-type shrinkage toward prior-date league means.
- `src/Python/pitcher_features.py`: league HR/FB prior smoothing (`lg_hr_fb_prior`) with explicit prior-strength blending.

This gives cold-start and small-sample rows a stable prior-date fallback while preserving leakage safety (no same-game outcomes in predictors).

6. Models

Feature design supplies the inputs; models convert those inputs into rate and exposure, then into count probabilities.

6.1 Strikeout-rate lane

Table 2. Active rate-model structure.

Component	Role
LightGBM (<code>sparse72_monotone</code>)	Ensemble member with monotone constraints
LightGBM (<code>final58</code>)	Ensemble member
Active blend (<code>0.60</code> / <code>0.40</code>)	Active production scorer (<code>sparse72_monotone</code> , <code>final58</code>)

The production decision is not a single-model claim; it is a governed ensemble choice validated across open-universe and deduped-manual lanes.

Table 2a. Current contender ``k_rate`` MAE (model-family lane, sparse-set run).

Table 2a is a challenger-screen table for single-model error behavior on shared sparse feature sets; it is not the deployment champion table.

Rank	Model family	Feature set	Mean <code>k_rate</code> MAE
1	ridge	<code>production_sparse72</code>	0.07668

Rank	Model family	Feature set	Mean k_rate MAE
1 (tie)	ridge	production_sparse72_monotone	0.07668
3	lightgbm	production_sparse72_monotone	0.07669
4	lightgbm	production_sparse72	0.07707
5	histgbr	production_sparse72	0.07721

The current ensemble-sweep ranking artifact does **not** include k_rate MAE columns; it is ranked on decision metrics (ROI/risk/market-skill). Therefore, ensemble k_rate MAE is reported as **not available in that artifact lane**. Also, family-model tags in this lane reflect a mixed budget (some default configs, some small inner-fold tuning), so rankings should be read as practical challenger screens rather than a perfectly equal hyperparameter-budget bakeoff.

Why Ridge can rank first in Table 2a and not be the deployment champion:

1. Table 2a is a **single-model k_rate error lane**.
2. Deployment championing is a **full decision lane** ($k_rate \times TBF \rightarrow P(K \geq L)$) with market-skill and risk metrics).
3. A tiny k_rate MAE edge does not guarantee better calibrated line probabilities or better realized risk-adjusted return after exposure, pricing, and bet-selection gates.

Note: Why ensemble over single model (paper/interview short form)

- **Single model = best point forecaster** in chronological MAE lanes.
- **Ensemble = best deployable decision engine** after calibration + market/risk governance.
- The project selects single-model leaders for challenger tracking and model-quality reference.
- The project selects deployment champions on decision metrics (skill vs market, ROI, Sharpe/Sortino, drawdown, CLV behavior).
- Therefore, “best MAE model” and “best deployed profile” can differ without contradiction.

6.2 Projected batters faced (TBF)

Rate alone is not a strikeout count. The second factor is projected batters faced: same-game PA is used only as a historical exposure oracle for training and evaluation, never as a predictor. Predictors include rest, lagged PA / Outs / Pitches, home/park/lineup K context, and thin team bullpen L1–L3d pitch/pitcher-use lookbacks (**24** features).

Frozen choice: Ridge with the thin bullpen feature set (coefficients persisted for reproducible scoring).

6.3 Count layer

The count layer is where rate and exposure become the paper’s target quantities, following the standard mean $\times$ exposure construction for count probabilities [3, 4]:

$$E[K] = k_rate_hat \times TBF_hat$$

$P(K \geq L)$ via Binomial / Poisson with $n = \text{round}(TBF_hat)$

Same-game PA never enters prop probabilities.

Notation used throughout Sections 6–8 is: model probability (p_model), de-vig market probability (p_market), edge ($p_model - p_market$), expected strikeouts ($E[K]$), and CLV in probability points (CLV_pp).

The production evaluation emphasis is now decision-lane quality (market-skill, replay ROI/risk path, and deployment robustness) rather than legacy internal MAE tables alone.

7. Ablations and feature-set freeze

The ablation framework now has two jobs: (a) quantify single-model sensitivity, and (b) provide candidate inputs for the ensemble governance lane.

7.1 Current protocol

- Outer chronology: anchored walk-forward windows.
- Inner chronology: model/feature tuning only inside training spans.
- Final rank surface: open-universe skill first, then deduped one-opportunity manual replay, then deployment checks.

7.2 Current takeaway

Current decisions are made on compact frozen sets (`sparse72_monotone`, `final58`) and their weighted ensemble behavior.

7.3 XGBoost monotone in the promotion workflow

XGBoost monotonic constraints were evaluated in follow-up parity runs. The active promotion workflow remains centered on the currently deployed LightGBM monotone stack because it carries the complete production artifact and governance contract.

1. the production monotone pathway (constraint mapping, validation, and artifact lineage) was already hardened around the LightGBM implementation,
2. the sparse-set challenger sweep prioritized chrono-safe comparability and decision-lane governance checks over expanding multiple monotone implementations at once,
3. adding XGBoost-monotone as a promoted lane would require its own constraint-sign audit and equal governance artifact contract before fair promotion.

The evidence claim is: **multiple model families were tested** on shared sparse datasets and chronological splits, and monotone behavior was checked in both LightGBM and XGBoost challenger lanes.

7.4 Aug 2026 follow-up parity checks (targeted reviewer questions)

To reduce ambiguity, a focused parity sweep was added after the manuscript rewrite:

1. enable XGBoost monotone constraints in the sparse-set family ablation runner,
2. run base-budget and tuned-small-budget comparisons on the same sparse sets and chronological folds,
3. bridge the MAE lane to decision-lane metrics in a separate governance replay comparison.

Key findings from the added artifact runs:

- **Ablation (base budget):** XGBoost unconstrained (`expected_K` MAE `1.8451`) and XGBoost monotone (`1.8511`) both trailed LightGBM and Ridge in this sparse-lane setup.
- **Ablation (tuned-small):** best XGBoost variants remained behind (`~1.8242` unconstrained, `~1.8352` monotone), while Ridge remained the MAE leader and LightGBM stayed closer to the top cluster.
- **Decision-lane bridge:** high MAE rank did not map one-to-one to best market/risk profile; best governance rows in the added replay comparisons were model-family dependent and changed with tuning budget, reinforcing lane separation.

These checks convert prior process rationale into direct evidence: XGBoost monotone was tested as a challenger, but did not clear the sparse-lane promotion bar in this cycle.

8. Full-stack evaluation

Component metrics are necessary but incomplete. Once rate and TBF are frozen, the object that must be judged is the full composition:

$k_rate \times TBF \rightarrow E[K] \rightarrow P(K \geq L)$

Table 3. Full-stack evaluation gates and current production results.

Evaluation gate	Result
Active deployment blend	0.60 <code>sparse72_monotone</code> / 0.40 <code>final58</code>

Evaluation gate	Result
Decision-lane ROI	0.4363
Decision-lane PnL	+24.17u (1u = 50 USD)
Decision-lane Sharpe	0.4352
Decision-lane Sortino	0.4277
Decision-lane Calmar	1.1841
Decision-lane max drawdown	0.3685
Market-skill deltas	+0.2069 Brier skill vs market, +0.1551 LogLoss skill vs market
Probability quality (active profile)	Brier 0.2090, LogLoss 0.6087, ECE 0.0639, MCE 0.1353
Execution controls	Board-to-ledger parity lock, quality gates, policy profile freeze KING_PROFILE_AUG2026

Calibration is summarized by expected calibration and scoring diagnostics [5, 6]. The active deployment profile reports ECE 0.0639, MCE 0.1353, and positive market-skill deltas versus market baseline, which is the paper's direct "ensemble versus books" evidence.

8.1 Metric hierarchy used in this manuscript

- `k_rate` MAE: single-model rate accuracy lane.
- `expected_K` MAE: full-stack point-forecast lane (`k_rate_hat` × `TBF_hat`).
- Brier/LogLoss skill vs market and ROI/risk metrics: deployment-governance lane.

Each lane answers a different question; winners are not interchangeable across lanes.

8.2 Backtest uncertainty and multiple-testing correction (active 26-bet lane)

The active audited manual lane contains $n=26$ graded recommendations (`top3`, floor 0.12). Because this sample is small, uncertainty and selection effects are reported explicitly.

Bootstrap percentile intervals (10,000 resamples):

- ROI 0.4363 with 95% CI [0.0337, 0.8072]
- Sharpe 0.4352 with 95% CI [0.0431, 0.9997]
- Sortino 0.4277 with 95% CI [0.0459, 0.7866]
- PnL +24.17u (1u = 50 USD) with 95% CI [+1.85u, +45.45u]

Date-block bootstrap (resampling by slate date to reduce same-day dependence assumptions) yields similarly wide intervals:

- ROI 95% CI [0.0084, 0.7739]
- Sharpe 95% CI [0.0885, 0.9296]
- Sortino 95% CI [0.0946, 0.7457]

Multiple-testing-aware Sharpe diagnostics (Bailey/López de Prado style):

- Probabilistic Sharpe Ratio (PSR, benchmark Sharpe 0): 0.9701
- Deflated Sharpe Ratio (DSR, trial-adjusted using $N=5161$ tested configurations): 0.0349

Interpretation: raw Sharpe is positive, but trial-adjusted significance remains weak at the current sample size and search breadth. Deployment claims are therefore framed as governed operational evidence rather than conclusive statistical dominance.

8.3 Slippage sensitivity (fixed 26-bet set)

To test execution fragility, adverse fill haircuts were applied to the same 26-bet audited set (no re-selection of bets).

Probability haircut (pp)	ROI	PnL (u, 1u=50 USD)	Sharpe	Sortino
0.0	0.4363	24.17	0.4352	0.4277

Probability haircut (pp)	ROI	PnL (u, 1u=50 USD)	Sharpe	Sortino
0.5	0.4313	23.89	0.4387	0.4206
1.0	0.4263	23.62	0.4336	0.4135
2.0	0.4163	23.06	0.4234	0.3997

The profile remains positive under this small haircut grid, but risk-adjusted metrics compress as expected.

8.4 Sample-size plan from DSR power targets

Holding observed return-shape moments and trial count fixed, the DSR power check implies approximate sample targets of:

- n ≈ 98 bets to reach DSR > 0.5,
- n ≈ 147 bets to reach DSR > 0.8.

At the current recommendation density (about one recommendation per slate day), this corresponds to roughly 72 to 121 additional graded recommendations beyond the current audited set.

For clarity of interpretation: Sharpe is mean excess return per unit total volatility, Sortino is mean excess return per unit downside volatility, and ROI is return over stake in the same evaluation lane.

Metric-purpose rule used in this manuscript:

- MAE/RMSE/R² claims come from chronological model-evaluation lanes (2023–2024 walk-forward/CV and 2025 holdout protocol).
- Open 2025–2026 and manual replay lanes are used for market/decision metrics (Brier/LogLoss skill vs market, ROI, Sharpe, Sortino, drawdown, CLV), not MAE promotion claims.

Figure 2. Reliability diagram for count-layer line probabilities. Points near the diagonal indicate well-calibrated bins; mean ECE ≈ 0.024 with no post-hoc recalibration.

These checks are confirmatory: they did not uncover large unused gains on the frozen stack’s internal metrics.

8.5 Production governance state (Aug 2026)

Current live operations use a compact three-lane governance model:

1. **Skill lane (open universe):** rank candidates on market-skill metrics across broad opportunity sets.
2. **Decision lane (manual, deduped):** enforce one-opportunity-one-bet fairness and evaluate realized risk/return behavior.
3. **Deployment lane:** transfer calibration and push the winning blend to live runtime with parity controls.

Key active controls:

- profile lock: KING_PROFILE_AUG2026,
- edge floor: 0.12 with line-aware side floors,
- isotonic probability calibration in production,
- fractional Kelly criterion is used as the sizing heuristic (reported here in 50 USD units),
- segmented line/price/maturity correction overlays,
- board-to-ledger parity reconciliation before governance status.

Failure-mode behavior in operations: if parity or quality gates fail, the promotion path is fail-closed (no automatic profile promotion) until reconciliation and gate re-clearance.

Current artifact-backed winners

- **Open-universe deduped sweep top profile:** 0.05 sparse72 / 0.45 sparse72_monotone / 0.50 final58
ROI 0.6612, Sharpe 0.9468, Sortino 0.6954, skill deltas +0.0825 Brier / +0.0645 LogLoss.

- **Active deployment champion (deduped transfer lane):** 0.60 sparse72_monotone / 0.40 final58
ROI 0.4363, PnL +24.17u (1u = 50 USD), Sharpe 0.4352, Sortino 0.4277, max drawdown 0.3685, skill deltas +0.2069 Brier / +0.1551 LogLoss.

Interpretation note: this profile generated 26 recommendations in the audited manual lane, approximately one recommendation per slate day over the sampled period.

Execution note: reported replay metrics assume fills at recorded replay prices; explicit slippage/transaction-cost haircuts should be applied in subsequent robustness updates.

This split keeps winner selection explicit: open-universe breadth ranking and deployment robustness are related but distinct optimization targets.

Larger settled-threshold view (operational, not the audited-26 lane).

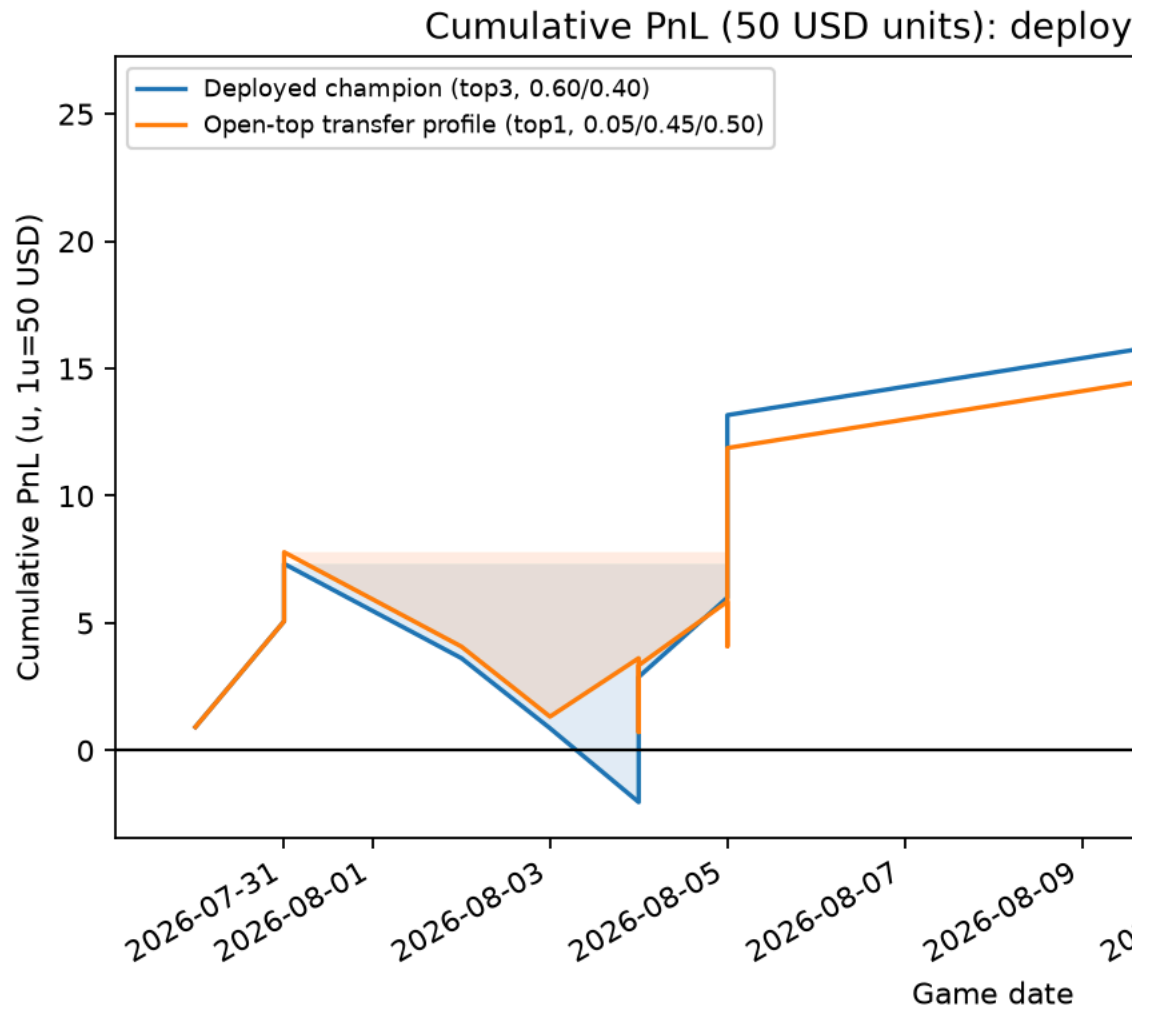
To calibrate floor behavior on a broader post-freeze operations sample, settled positive-stake rows from 2026-07-31 forward were bucketed by policy floor. This is an execution-policy diagnostic lane (volume/ROI tradeoff), not a replacement for the 26-bet audited manual lane above.

Policy	Bets (n)	ROI
Single floor 0.08	307	0.0388
Single floor 0.10	285	0.0485
Single floor 0.12	265	0.0710
Single floor 0.14	209	0.0744
Single floor 0.16	159	0.0933
Dual floor (over=0.10 , under=0.08)	299	0.0421

Interpretation: tighter floors reduce volume and improve realized ROI in this window; the active 0.12 deployment floor remains the current balance point between throughput and edge quality.

Working hypothesis for edge persistence: strikeout-prop markets are thinner and adjust less uniformly than major side/total markets, so leakage-safe pitcher-form and lineup-context features can remain underpriced at some times of day. This is a practical market-microstructure hypothesis, not a proof of persistent inefficiency.

Figure 3. Cumulative PnL overlay for the deployed champion (top3) and open-top transfer profile (top1) at floor 0.12, shown in 50 USD units. The overlay highlights the breadth-versus-robustness tradeoff described in this section.



8.6 Interpretability and model-driver evidence (artifact-backed)

Interpretability is handled here as **stable directional evidence** from leakage-safe ablations and challenger parity runs, rather than as one-off global importance ranks.

Primary evidence used in this manuscript:

- nested family/window ablation records (docs/research/historical-step-findings-summary.md and linked artifacts/feature_research/*ablation* outputs),
- sparse-lane model-family parity artifacts (artifacts/model_quality/sparse72_model_family_ablation/aug21_parity_base/ and aug21_parity_tuned_small/),
- governance-lane bridge artifacts (artifacts/odds_log/model_family_governance_compare_aug21_governance_base.csv and ..._tuned_small.csv).

Driver-level interpretation used for interviews and operational review:

- Point-forecast lane:** Ridge remains the best single-model MAE challenger (expected_K MAE about 1.7621, k_rate MAE about 0.07668) on the sparse-lane parity contract.
- Monotone-rate lane:** LightGBM monotone remains in the near-frontier cluster (for example expected_K MAE about 1.7689 in base-budget parity) while preserving the production monotone policy path.

- **Challenger variation:** XGBoost was evaluated in both unconstrained and monotone forms; both trailed current sparse-lane leaders in this cycle (tuned-small examples: about 1.8242 unconstrained and 1.8352 monotone expected_K MAE).
- **Deployment implication:** MAE rank and deployment rank diverge by design; governance winners are selected on market-skill and risk metrics, not MAE alone.

8.7 Start-level case narratives (audited 26-bet lane)

The audited lane (top3, floor 0.12) contains both high-edge confirmations and misses. The purpose of these examples is to show **decision behavior under uncertainty**, not to re-argue MAE.

- **Right-call example (high edge, under):** 2026-07-31 Paul Skenes under 7.5 (edge=0.2742) settled as a win (rpd=+1.10, positive CLV).
- **Right-call example (high edge, over):** 2026-07-30 Roki Sasaki over 5.5 (edge=0.2293) settled as a win (rpd=+1.00).
- **Wrong-call example (high edge, under):** 2026-08-15 Ian Seymour under 6.5 (edge=0.2292) settled as a loss (rpd=-1.00) despite positive pre-bet edge and positive CLV.

These cases illustrate the practical pattern seen across the lane: edge/CLV signal can be directionally useful while individual outcomes remain noisy at start level.

8.8 Operational benchmark snapshot (local workstation)

To make the MLE-facing reliability claims auditable, this manuscript records concrete runtime slices from the parity and governance workflow used in the Aug 2026 checkpoint:

- sparse-lane base parity run (ablate_sparse72_model_families.py): about 234s,
- sparse-lane tuned-small parity run: about 797s,
- governance bridge run (compare_model_family_governance.py, base): about 120s,
- governance bridge run (tuned-small): about 235s.

End-to-end parity-plus-governance refresh for that checkpoint was about **23 minutes** wall-clock on the local workstation.

Operational controls remain fail-closed: if parity/quality gates fail, promotion does not proceed until reconciliation clears (production/ops/build_validation_ops_report.py, production/ops/build_policy_governance_report.py).

9. Limitations

Population scope. Reported core metrics use the $PA \geq 9$ cohort and therefore describe conventional-length starts.

Game-level variance ceiling. Even with modern features and ensemble blending, pitcher-game outcomes retain substantial irreducible variance.

Exogenous signal coverage. Some high-impact context channels (weather micro-effects, travel fatigue, umpire framing) remain outside the frozen production feature set.

Statistical confidence under small audited lane. The active manual lane ($n=26$) yields wide intervals on ROI and risk ratios, and trial-adjusted Sharpe significance is limited at current sample size.

Interpretability breadth. This version includes artifact-backed family/parity evidence and start-level narratives, but it does not yet include a full SHAP/conditional-permutation atlas on every frozen deployment profile.

Operational stress breadth. Runtime slices are now documented for the parity/governance checkpoint workflow, but continuous production SLO tracking (for example multi-month p95 refresh latency and rollback-time distribution) is still open.

10. Conclusion

This work delivers a leakage-safe pregame strikeout system that now behaves like a quant production stack: compact frozen feature sets, ensemble rate scoring, TBF exposure modeling, calibrated count probabilities, and governed execution controls.

The core engineering result is not just lower error versus simple baselines; it is a reproducible operating workflow where model selection, policy thresholds, and daily execution are linked through auditable artifacts and chronological validation.

Quant-honesty checks in this version show positive raw performance, but deflated-Sharpe evidence remains low ($\text{DSR}=0.0349$), so additional live sample accumulation is required before claiming durable statistical edge.

Reproducibility statement

Code for the leakage-safe pipeline, nested selection utilities, trainers, and count layer lives in the public repository: <https://github.com/Cbkaplinger/MLB-Props> (Python ≥ 3.11 ; Polars for feature construction; scikit-learn and LightGBM for models; pytest for automated leakage and pipeline checks). Experiments reported here were run on a local Windows workstation with a project-local virtual environment. Primary data are pitch-level Statcast exports accessed via Baseball Savant (commonly retrieved with community tooling such as pybaseball); users should respect Baseball Savant / MLB terms of use for redistribution and commercial use. Generated local artifacts (model binaries, fold summaries, and governance outputs) are reproducible from the documented runners but are not required to read the manuscript's tables and figures.

At the Aug 2026 freeze checkpoint, operator and research notebook surfaces were re-executed end-to-end after policy and calibration updates, with successful execution recorded in the notebook execution artifacts under `artifacts/odds_log/`.

Data Availability

Statcast data can be retrieved per-user via public tools (e.g., pybaseball) rather than redistributing bulk parquet, which helps sidestep Baseball Savant / MLB terms-of-use concerns around bulk redistribution.

Appendix A. Repository map

Internal repository names, paths, and frozen-artifact identifiers are collected here so the body text can stay narrative. They do not change any metric reported above. Repository cleanup governance and keep/hold/delete audit protocol are maintained in `docs/reference/repo_canonical_map.md` and `docs/reference/repo_waste_sweep_checklist.md`, with the latest pass report at `docs/reference/reports/repo_quality_passthrough/2026-08-21.md`.

A.1 Feature-set aliases

Table A1. Feature-set aliases used in code.

Alias	Size	Description
<code>production_sparse72</code>	72	Compact sparse baseline
<code>production_sparse72_monotone</code>	72	Sparse baseline with monotone constraints
<code>production_final58_consensus</code>	58	Consensus-pruned compact challenger
<code>ridge_vif</code>	73	Linear-model research companion
<code>workload_context_bullpen</code>	24	Frozen TBF feature set (thin bullpen)

A.2 Frozen model artifacts

Frozen model stems, model sidecars, and artifact hashes are maintained in the repository model card and the locked comparison-pack manifest:

- `docs/reference/model-card.md`
- `docs/reference/reports/model_comparison_pack/2026-08-21.manifest.json`

Generated research outputs under `artifacts/` are local/reproducible and typically gitignored.

A.3 Documentation and code map

Table A3. Documentation and code map.

Topic	Location
Governance metric lanes	<code>docs/reference/governance_metric_stack.md</code>
Model card	<code>docs/reference/model-card.md</code>
Canonical production runbook	<code>production/README.md</code> , <code>production/RUNBOOK.md</code>
Feature/pipeline implementation notes	<code>docs/reference/dev-notes.md</code> , <code>src/Python/</code>
Rate training	<code>models/Strikeout-Model/train.py</code>
TBF training	<code>models/TBF-Model/train.py</code>
Count layer findings	<code>docs/research/count_layer_findings.md</code>
Walk-forward quality gates	<code>docs/research/phase11_model_quality_gates.md</code>
Live assembly plan	<code>docs/reference/live_assembly_plan.md</code>
Research chronology (historical log)	<code>docs/research/PAPER_NOTES.md</code>

A.4 Quant-governance artifacts (Aug 2026 expansion)

Raw governance artifact filenames are maintained in the repository documentation instead of duplicated in the manuscript body:

- `production/README.md`
- `docs/reference/model-card.md`
- `docs/reference/reports/model_comparison_pack/2026-08-21.md`
- `docs/reference/reports/model_comparison_pack/2026-08-21.manifest.json`

Three-lane methodology now used in governance:

1. **Skill lane (open universe):** rank candidates on market-skill metrics over large opportunity sets.
2. **Decision lane (manual, deduped):** enforce one-opportunity-one-bet fairness and evaluate realized quant path metrics.
3. **Deployment lane (transfer + runtime):** transfer calibration from open panel to manual lane, then deploy via config-driven live scorer.

A.5 Active production profile snapshot

Table A5. Current deployment profile from

open_top3_transfer_manual_replay_aug21_deduped_top3_from_dedupedsweep.json.

Metric	Value
Blend weights	0.60 sparse72_monotone / 0.40 final58
Edge floor	0.12
Bets	26
Stake	55.40u (1u = 50 USD)
PnL	+24.17u (1u = 50 USD)
ROI	0.4363
Sharpe / Sortino / Calmar	0.4352 / 0.4277 / 1.1841
Max drawdown	0.3685
CLV mean (pp)	0.0252
Positive CLV share	0.70
Brier / LogLoss	0.2090 / 0.6087
ECE / MCE	0.0639 / 0.1353
Market skill deltas	+0.2069 Brier / +0.1551 LogLoss

Consistency note on ablation tables.

k_rate MAE contender comparisons come from sparse-set ablation artifacts, including the parity snapshots at artifacts/model_quality/sparse72_model_family_ablation/aug21_parity_base/ablation_summary_ranked.csv and artifacts/model_quality/sparse72_model_family_ablation/aug21_parity_tuned_small/ablation_summary_ranked.csv. Deployment-king tables come from deduped replay/transfer artifacts and are ranked on decision metrics, not k_rate MAE.

References

1. Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T.-Y. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
2. Hoerl, A. E., and Kennard, R. W. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970.
3. Cameron, A. C., and Trivedi, P. K. *Regression Analysis of Count Data*. Cambridge University Press, 2nd edition, 2013.
4. Hilbe, J. M. *Negative Binomial Regression*. Cambridge University Press, 2nd edition, 2011. (Also discusses overdispersion relative to the Poisson/binomial mean–variance relationship; beta-binomial used here as a related overdispersion check.)
5. Naeini, M. P., Cooper, G. F., and Hauskrecht, M. Obtaining well calibrated probabilities using Bayesian binning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2015.
6. Guo, C., Pleiss, G., Sun, Y., and Weinberger, K. Q. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017.

7. Tango, T. M., Lichtman, M. G., and Dolphin, A. E. *The Book: Playing the Percentages in Baseball*. Potomac Books, 2007. (FIP lineage and defense-independent pitching intuition.)
8. Slowinski, P. xFIP. FanGraphs Library / glossary documentation. URL: <https://library.fangraphs.com/pitching/xfip/> (accessed 2026-07-28). (xFIP replaces HR with expected HR via fly-ball rate $\times$ league HR/FB.)
9. Tango, T. M. Marcel the Monkey Forecasting System. Tangotiger / Hardball Times documentation of the minimal season projection baseline (weighted recent seasons, regression to the mean, age adjustment). URL: <https://www.tangotiger.net/marcel/> (accessed 2026-07-28).
10. Silver, N. Introducing PECOTA. In Huckabay, G., Kahrl, C., Pease, D., et al. (Eds.), *Baseball Prospectus 2003*. Brassey's, 2003, pp. 507–514.
11. Bergmeir, C., Hyndman, R. J., and Koo, B. A note on the validity of cross-validation for evaluating autoregressive time series prediction. *Computational Statistics & Data Analysis*, 120:70–83, 2018.